

## TGDetect — Heterogeneous Continuous-Time TGNN Architecture

In-Depth Architecture Study · Multi-Step Cyberattack Detection Framework  
Aligned with Objectives O1–O4

## LAYER 1 — MULTI-SOURCE DATA INGESTION &amp; PREPROCESSING

## OBJ 1

## 1.1 Heterogeneous Log Source Collection

Authentication Logs System Audit Logs Network Flow Data DNS Query Logs Cloud API Logs

**Entity Types:** Users, Processes, Files, IP Addresses, Domains, Ports, Cloud Resources, Registry Keys

**Event Types:** Process Creation, File Read/Write, Network Connection, DNS Resolution, Authentication, Privilege Escalation, API Call

## 1.2 Temporal Normalization &amp; Entity Resolution

## NORMALIZATION PIPELINE

Raw Timestamps → UTC Standardization → Continuous-Time →  $t \in \mathbb{R}^+$

**Entity Resolution:** Deduplication via hashing (SHA-256 of entity attributes). Cross-source identity linking (e.g., PID-to-User mapping, IP-to-Domain correlation).

## SCHEMA UNIFICATION

Unified Event Schema  
(src, dst, action, t, attrs)

**Feature Extraction:** Categorical encoding of action types, numerical features from event attributes, temporal features from timestamp deltas.



## LAYER 2 — HETEROGENEOUS TEMPORAL GRAPH CONSTRUCTION

## OBJ 1

## 2.1 Graph Schema Definition

## Nodes (Entities)

Each unique entity becomes a node with type-specific initial feature vector. **Heterogeneous**

## Edges (Interactions)

Each event becomes a directed, timestamped edge connecting source to destination entity.

## Formal Graph Definition

$G_t = (V, E, T, A)$  where  $V$  = set of heterogeneous nodes,  $E$  = set of directed edges,  $T: E \rightarrow \mathbb{R}^+$  assigns continuous timestamps,  $A: E \rightarrow \mathbb{R}^k$  assigns edge feature vectors.

## Node Features

$h_v^{(0)}$  — Type-specific initial embedding. Dimension:  $d$

## Edge Features

$e_{uv}$  — Action type encoding + attribute vector. Includes timestamp  $t_{uv}$

## 2.2 Continuous-Time Temporal Graph Construction

## EDGE TEMPORAL ORDERING

Events are indexed as a chronologically sorted sequence. Each new event  $e_i = (u, v, t_i, a_i)$  is appended to the temporal edge list, maintaining the invariant  $t_1 \leq t_2 \leq \dots \leq t_n$ .

**Sliding Window:** A configurable temporal window  $\Delta W$  retains only recent edges for online inference, preventing unbounded memory growth.

## GRAPH STORAGE

## In-Memory Graph

CSR format adjacency + timestamp array for fast temporal neighborhood queries.

## Persistent Storage

Neo4j / NetworkX serialization for full provenance graph persistence.



## LAYER 3 — HETEROGENEOUS CONTINUOUS-TIME TGNN CORE ENGINE

## OBJ 1

## 3.1 Continuous-Time Time Encoding Module

## Sinusoidal Time Encoding

Projects scalar timestamps into high-dimensional space preserving temporal proximity relationships.

## Time Decay Function

Exponential decay weights recent interactions higher than distant ones.

## Temporal Attention Weight

$\alpha_{uv}^{(t)} = \exp(-\sigma \cdot |t - t_{uv}|) \cdot \text{softmax}(W_a \cdot [h_u \parallel h_v \parallel TE(t_{uv})])$  — Combines temporal recency with learned attention over node states and time embeddings.

## Sinusoidal Time Encoding

$TE(t) = [\sin(\omega_1 t), \cos(\omega_1 t), \dots, \sin(\omega_d t), \cos(\omega_d t)]$

where  $\omega_k = 1 / 10000^{2k/d}$  (positional encoding frequencies)

## 3.2 Neighborhood Message Passing (Temporal Attention)

Source Node  $h_u$  → Linear Transform  $W_\phi$  → Temporal Attention  $\alpha$  → Weighted Aggregate  $\Sigma$  → Message  $\tau$

**Multi-Head Attention:**  $K$  independent attention heads compute parallel message representations, concatenated and projected:  $m_v = \parallel_{k=1}^K W_k \cdot \text{Attn}_k(N_\phi(v))$

## Message Computation

$m_v^{(t)}(t) = \sum_{u \in N_\phi(v)} \alpha_{uv}^{(t)}(t) \cdot \phi(h_u^{(t)} \parallel e_{uv} \parallel TE(t_{uv}))$

$N_\phi(v)$  = temporal neighborhood of  $v$  at time  $t$  (all edges  $(u, v, t')$ :  $t' \leq t$ )

## 3.3 Temporal Aggregation Module

## Temporal Attention Pooling

Attention-weighted aggregation over all timestamps in the neighborhood window. Learns which time steps matter most for the current classification.

## RNN/GRU Temporal Encoder

Sequentially processes chronologically ordered messages through a GRU cell, capturing temporal dynamics and event ordering effects.

## Node State Update

$h_v^{(t+1)}(t) = \text{GRU}(h_v^{(t)}(t), \text{AGG}(\{m_u^{(t)}(t) : t_i \in [t - \Delta W, t]\}))$  — Updates node embedding by aggregating temporally-weighted messages from neighbors.

## Sliding Window Aggregation

Aggregates messages within a configurable time window  $\Delta W$ , enabling efficient online processing without recomputing full history.

## 3.5 Heterogeneous Type-Aware Processing

## Type-Specific Projection

Separate linear projections  $W_t$  for each node/edge type  $t$ , enabling type-aware feature transformation.

## Type Embedding

Learnable type embeddings  $e_t$  concatenated to node features, providing structural type information to the model.

## Cross-Type Attention

Attention mechanism that modulates message strength based on source-target type pair, capturing cross-type interaction significance.

## Type-Aware Message Function

$$\Phi_{(u),t(u);(v),t(v)}(h_u, e_{uv}, TE(t)) = W_{\Phi(t)} \cdot \text{MLP}(W_{\Phi(t)} h_u \parallel W_{\Phi(t)} e_{uv} \parallel TE(t_{uv}) \parallel e_{\Phi(t)} \parallel e_{\Phi(v)})$$

## 3.6 Classification Head

Final Embedding  $h_v^{(L)}(t) \rightarrow$  MLP Decoder  $\rightarrow$  Softmax / Sigmoid  $\rightarrow$  Attack Label  $\hat{y}$

**Binary Classification:**  $\hat{y} = \sigma(W_{out} \cdot h_v^{(L)}(t) + b)$  — Malicious vs. Benign event prediction.

**Multi-Class:** APT, LoT, Zero-day, Benign — Cross-type attack categorization with hierarchical classification.

**Output:** Event-level classification with confidence score, plus node-level anomaly scores for graph-wide detection.



## LAYER 4 — ONLINE CONCEPT DRIFT ADAPTATION

## OBJ 2

## 4.1 Drift Detection Engine

## Statistical Drift Detector

ADWIN (Adaptive Windowing) or Page-Hinkley test monitors prediction error distribution for significant changes.

## Feature Distribution Monitor

KL divergence / KS test compares incoming feature distributions against reference window statistics.

## Drift Detection Condition

$H_0: D(P_{ref}, P_{new}) \leq \epsilon$  (null hypothesis: no drift)

$H_1: D(P_{ref}, P_{new}) > \epsilon$  (drift detected)

When  $H_1$  accepted  $\rightarrow$  trigger adaptation pipeline.

## 4.2 Incremental Model Adaptation

Drift Signal  $\rightarrow$  Buffer Recent Labeled Data  $\rightarrow$  Selective Replay  $\rightarrow$  Fine-Tune TGNN  $\rightarrow$  EWC Regularization  $\rightarrow$  Updated Model

## ELASTIC WEIGHT CONSOLIDATION (EWC)

**Prevents Catastrophic Forgetting:** Computes Fisher Information Matrix  $F$  on old data. Adds penalty  $\lambda \sum_i F_i (\theta_i - \theta_i^*)^2$  to loss, constraining important parameters from changing too much.

## EXPERIENCE REPLAY BUFFER

**Maintains Old Knowledge:** Reservoir-sampled buffer of representative past events interleaved with new data during fine-tuning. Buffer size  $B$  with FIFO replacement.

$\rightarrow$  Continuous Feedback Loop: Detection Accuracy Monitor  $\rightarrow$  Drift Detection  $\rightarrow$  Model Update  $\rightarrow$  Accuracy Re-evaluation  $\rightarrow$  (repeat)



## LAYER 5 — ATTACK BACKTRACKING &amp; PATH RECONSTRUCTION ENGINE

## OBJ 3

## 5.1 Alert-Triggered Backtracking

Detection Alert  $\hat{y}_t(t) \rightarrow$  Identify Alert Node  $\rightarrow$  Reverse BFS/DFS  $\rightarrow$  Temporal Path Scoring  $\rightarrow$  Attack Chain

## Temporal Subgraph Extraction

Extracts the connected temporal subgraph  $G_t$  rooted at the alert node, traversing backward through temporal edges.

## Attention-Based Path Ranking

Uses TGNN attention weights  $\alpha_{uv}$  to rank edges by contribution to the malicious classification.

## Path Significance Score

$$S(\text{path}) = \sum_{e_i \in \text{path}} \alpha_{e_i} \cdot P(\text{malicious} | e_i) \times \text{temporal\_coherence}(\text{path})$$

Paths with  $S(\text{path}) > \text{threshold } \tau$  are included in the reconstructed attack chain.

## 5.2 Multi-Step Attack Scenario Reconstruction

## Initial Compromise Identification

Finds the earliest node in the attack chain — the point of initial entry (e.g., phishing email, exploited vulnerability).

## Lateral Movement Tracing

Traces the attacker's movement across the network: which machines were compromised, what credentials were used, what data was accessed.

## Impact Assessment

Identifies all affected entities (files, credentials, machines) and the blast radius of the attack for IR prioritization.

**Output Format:** Chronologically ordered attack timeline with entities, actions, timestamps, and confidence scores for each step in the chain.

**Visualization:** Interactive temporal graph visualization (pyvis/Plotly) highlighting the attack path in red, normal activity in gray.



## LAYER 6 — TEMPORAL EXPLAINABILITY VIA MITRE ATT&amp;CK MAPPING

## OBJ 4

## 6.1 Attention Weight Extraction &amp; Temporal Mapping

TGNN Attention Weights ( $\alpha_{uv}$ )  $\rightarrow$  Top-K Edge Selection  $\rightarrow$  Temporal Ordering  $\rightarrow$  Behavioral Pattern Extraction

**Attention Aggregation:** Per-edge attention scores are aggregated per entity pair and time window to identify the most influential interactions contributing to the classification.

**Temporal Pattern:** Extracts **when** (timestamp progression), **how** (action sequence), and **why** (attention-weighted significance) each event contributed.

## 6.2 MITRE ATT&amp;CK Framework Mapping

## Tactic-Level Mapping

Maps detected behavior sequences to MITRE ATT&CK tactics (Initial Access, Execution, Persistence, Privilege Escalation, Lateral Movement, Defense Evasion).

## OUTPUT ANNOTATIONS

Tactic Taxonomy

Technique Taxonomy

